Übungsblatt 8: HSEM, Watchdogs und Fortschrittsüberwachung

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: HSEM schützt den vollständigen Zugriff (8 Punkte)

1. Ein Shared-Zähler steht bei 71. Beide Kerne lesen 71, erhöhen privat und schreiben jeweils 72. Geben Sie tatsächlichen und erwarteten Endwert an. Warum genügt ein HSEM nur um den Store nicht? (2 Punkte)
2. Beide Kerne haben je 80 Versuche; M7 meldet 76, M4 79 erfolgreiche vollständig geschützte Inkremente ab null. Welcher Endwert und welche Fehlversuchszahlen sind zu erwarten? Was zeigt ein Endwert 153? (3 Punkte)
3. Zwei Tasks desselben Kerns benutzen `HAL_HSEM_FastTake` für dieselbe ID. Warum ist das kein Ersatz für einen lokalen Mutex? Nennen Sie die Rolle der ProcessID und eine passende lokale/globale Sperrregel. (3 Punkte)

Musterlösung: Tatsächlich 72, erwartet 73. Ein Schloss nur beim Schreiben verhindert nicht die veralteten Lesegrundlagen; Lesen, Ändern und Schreiben gemeinsam schützen. Erfolgssumme $76 + 79 = 155$, Fehlversuche vier und eins. Endwert 153 verletzt bei gültigem Abschluss die Bilanz um zwei. FastTake benutzt ProcessID null; zwei Tasks desselben Kerns erscheinen damit als dasselbe Besitzpaar, ohne Taskmutex und Rekursionszählung. Lokalen Mutex vor HSEM benutzen, kurze Bereiche, globale einheitliche Erwerbsordnung. HSEM erzeugt weder Cachewartung noch Taskprioritätsvererbung.

Aufgabe 2: Aufwecken innerhalb einer Gesamtfrist (8 Punkte)

M4 scheitert beim Take; M7 gibt frei, bevor M4 seine Notification scharfschaltet. Ein gespeichertes Thread-Flag wird erst bei einer scharfgeschalteten Freigabemeldung gesetzt. Genau ein Warter, keine weitere garantierte Freigabe.

1. Erklären Sie das Lost Wakeup und geben Sie eine richtige Reihenfolge von Notification, Take, Wait mit Restzeit und erneutem Take an. Beachten Sie, dass die Referenz-HAL ausgelöste Notification-Bits deaktiviert. (4 Punkte)
2. Gesamtbeginn 5 ms, Budget 18 ms. Meldungen kommen bei 11 und 20 ms, ein weiterer Take scheitert bei 14 ms. Berechnen Sie Deadline und Restzeiten. Warum nicht jedes Mal wieder 18 ms warten? (2 Punkte)
3. Zwei Kerne halten HSEM 2 bzw. 3 und fordern jeweils die andere ID. Wie verhindern Sie den Zyklus, und was muss ein Timeoutpfad mit bereits gehaltenen Schlössern tun? (2 Punkte)

Musterlösung: Die Freigabe kann vor der Meldebereitschaft verschwinden. Mit gültigem Handle: alte eigene Ereignisse klären, Notification aktivieren, Take versuchen. Bei Fehlschlag das inzwischen gespeicherte Flag nicht vor Wait löschen. Mit verbleibendem Gesamtbudget warten; nach Meldung erneut scharfschalten und Take versuchen. Die Meldung verleiht keinen Besitz; nur zulässige nichtblockierende ISR-API bei erlaubter Priorität benutzen. Deadline $5 + 18 = 23$ ms; Rest bei 11/14/20 ms: 12/9/3 ms. Neue volle Intervalle verlängern den Vertrag. Globale Ordnung 2 vor 3 verhindert den Kreis; bei Abbruch eigene Schlösser passend freigeben und Teilzustände behandeln.

Aufgabe 3: Watchdogfenster selbst berechnen (8 Punkte)

WWDG-Lehrmodell: PCLK 80 MHz, Vorteiler vier, Reload 120, Fenster 86. Erlaubt ist $63 < T \leq 86$, EWI bei 64, Reset beim Schritt nach 63. Erster Abwärtsschritt einen vollen Tick nach Start.

1. Berechnen Sie Tick, Fensteröffnung, EWI- und Resetzeit mit $\Delta t = 4096P/f$. (3 Punkte)
2. Bewerten Sie Refresh nach 4, 8 und 14 ms. Ist Gleichheit mit dem Fensterwert erlaubt? (1 Punkt)
3. IWDG ohne Fenster: Reload 749, Teiler 16; nur didaktisch LSI zwischen 30 und 42 kHz. Bestimmen Sie früheste/späteste Ablaufzeit mit $(R + 1)P/f$. Welche Grenze zählt für den Refresh? (2 Punkte)
4. Für den EWI verbleibt nominal ein WWDG-Tick; Eintritt 25 µs, Grundarbeit 18 µs, Reserve 22 µs. Wie viel Zusatzarbeit passt hinein, und weshalb ist EWI keine garantierte letzte Rettung? (2 Punkte)

Musterlösung: Tick 0,2048 ms. Öffnung nach 34 Schritten bei 6,9632 ms; EWI nach 56 Schritten bei 11,4688 ms; Reset nach 57 Schritten bei 11,6736 ms. 4 ms zu früh, 8 ms im Idealmodell gültig ($T = 81$), 14 ms nach bereits erfolgtem Reset. Gleichheit $T = 86$ erlaubt, praktische Reserve nötig. IWDG: $750 \cdot 16 = 12000$; frühestens 285,714 ms bei 42 kHz, spätestens 400 ms bei 30 kHz. Früheste Grenze mit Reserve maßgeblich; Werte sind keine Datenblattgrenzen. EWI-Zusatzbudget $204{,}8 - 25 - 18 - 22 = 139{,}8$ µs. Maskierung oder Verdrängung können den Callback verhindern; zu früher Refresh kann ohne vorherigen EWI resetten.

Aufgabe 4: Fortschritt und Grenzen der LED-Anzeige (8 Punkte)

1. Jetzt 50 ms: neue vollständige Nachweise von A bei 44 ms und B bei 29 ms, Frischefristen 9 und 18 ms. Wer ist frisch? Beweist ein laufender Idle-Thread Workerfortschritt? (2 Punkte)
2. Disjunkte Reaktionsgrenzen ab Stillstand: Frischefrist 24 ms, Abfrageverzug 4 ms, Meldeverzug 3 ms, Watchdogrest 12 ms, Bootreaktion 6 ms. Reicht die Grenze für 45 ms Anforderung? (2 Punkte)
3. Eine passende No-Access-MPU-Region schützt M7. Gilt das für dessen normalen privilegierten IRQ, für M4 und für DMA? Muss ein Arrayüberlauf innerhalb einer erlaubten Region faulten? (2 Punkte)
4. Eine Board-LED wird vor einem Reset ausgeschaltet. Beweist dies einen garantiert inaktiven externen Freigabeausgang während Reset und Boot? Welche zusätzlichen Eigenschaften müsste ein solcher Entwurf nachweisen? Keine externe Schaltung ist aufzubauen. (2 Punkte)

Musterlösung: Alter A sechs ms: frisch; B 21 ms: veraltet. Idle beweist nur seinen eigenen Fortschritt. Gesamtgrenze $24 + 4 + 3 + 12 + 6 = 49$ ms, größer als 45 ms. Normale privilegierte M7-Handler unterliegen der passenden aktiven MPU-Region, M4 und DMA nicht. Ein Zugriff außerhalb des C-Arrays innerhalb erlaubter Adressen muss keinen MPU-Fault auslösen. Ein vorheriger LED-Schreibwert beweist keinen Pegel bei hochohmigem Resetpin und keinen sicheren Bootpfad. Ein echter Freigabeentwurf braucht definierten elektrischen Resetpegel, inaktiven Start, überwachte Freigabe und begründeten Fehlervertrag; die Boardanzeige ist dafür kein Sicherheitsnachweis.

Aufgabe 5: Ein Wiedererwerb kann zu spät sein

Ein Ablauf mit 18 ms Gesamtbudget beginnt bei Tick `0xFFFFFFFA`. Jetzt ist Tick `0x00000004`. Wie viel Restzeit bleibt? Darf ein Wiedererwerb bei abgelaufenem Budget als rechtzeitiger Erfolg gezählt werden, nur weil das Schloss inzwischen frei ist?

Musterlösung: Vorzeichenlose Differenz zehn Ticks, Rest acht bei durchgehend 1-kHz-Tick und kurzer Distanz. Bei abgelaufenem Gesamtbudget ist der Zeitvertrag bereits verletzt; ein späterer Take ist kein rechtzeitiger Erfolg. Unbeobachtete Mehrfachumläufe werden durch Moduloarithmetik nicht erkannt.

Aufgabe 6: Einen Fortschritts-Supervisor implementieren

Programmieren, etwa 25–35 Minuten. Die CM4-Basis `lab08_health.c/.h` startet zwei Worker und einen Supervisor. Worker A aktualisiert seinen Nachweis ungefähr alle 60, B alle 100 Kernelticks. Der Supervisor kopiert unter einem Mutex Zeitstempel und Gültigkeitsbits; der Tick läuft mit 1000 Hz. Halten von **B1/PC13** unterbindet B-Nachweise, ohne dessen Thread oder den Scheduler zu stoppen.

1. Implementieren Sie `Lab08_Healthy(snapshot)` in `health_student.c`: Beide Worker müssen mindestens einen vollständigen Nachweis geliefert haben. A darf höchstens 80, B höchstens 140 Tickwerte alt sein. Nutzen Sie vorzeichenlose Zeitdifferenzen und behandeln Sie einen Nullpointer als ungültig.
2. Die Basis ruft Ihre Funktion alle ungefähr 20 Ticks auf. **LD1/PB0** zeigt gesund, **LD3/PB14** fehlenden/veralteten Nachweis, **LD2/PE1** Supervisoraktivität. Prüfen Sie Start, längeres Halten von B1 und Freigeben des Tasters. Wie kann der Supervisor weiterlaufen, während B fachlich nicht fortschreitet?
3. Prüfen Sie auf Papier die Grenze genau 80/140 und einen Zählerüberlauf. Warum müssen Snapshot und Uhr denselben Kerneltakt benutzen?
4. Ergänzen Sie als Erweiterung eine bewusst spezifizierte Fehlerverriegelung: Nach einem erkannten veralteten Nachweis bleibt die Fehleranzeige bis zum Reset aktiv. Die Startphase ohne erste Nachweise darf diese Verriegelung noch nicht auslösen.

Der Hardware-Watchdog bleibt in diesem Versuch ausgeschaltet. Sie implementieren und prüfen die spätere Refresh-Voraussetzung; die LED zeigt keine echte Watchdogfütterung und keinen sicherheitsgerichteten Ausgang.

Vorbereitete Basis: Projektanleitung, STM32CubeMX und VS Code mit STM32 VS Code Extension für **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Board und USB/ST-LINK genügen. Eigene Implementierung und tatsächlichen Prüfstatus festhalten; Debuggerdiagnosen bei angehaltenem zuständigem Kern lesen.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS und gegebenenfalls CMSIS-RTOS2. Papierwerte sind Lehrannahmen.

Musterlösung:

Musterlösung zu Aufgabe 6

Die Frischeprüfung benutzt die zur tatsächlichen Workeraktualisierung gehörenden Zeitstempel, nicht nur ein nachträglich beim Empfänger gesetztes Empfangsdatum. Mutex und Kopie liegen in der Basis.

```
#include "lab08_health.h"

int Lab08_Healthy(const Lab08_Health *s)
{
    return s != NULL && s->have_a && s->have_b &&
        (uint32_t)(s->now - s->last_a) <= 80U &&
        (uint32_t)(s->now - s->last_b) <= 140U;
}
```

Gleichheit ist frisch. `now=0x20`, `last_a=0xFFFFFFF0` ergibt Alter 48. Es werden kurze Beobachtungsabstände ohne mehrfache unbeobachtete Umläufe vorausgesetzt. Eine Anzeige beim ersten Supervisorlauf kann bis zu den ersten Nachweisen vorübergehend Fehler zeigen. B bleibt bei gedrücktem Taster ausführbar und wartet regelmäßig, liefert aber keinen fachlichen Fortschritt. Nach Freigabe erholt sich die Basisprüfung beim nächsten B-Nachweis.

Erweiterung: lokale statische Verriegelung erst setzen, wenn beide `have`-Bits gültig sind und die Frischeprüfung fehlschlägt. Danach bis Reset immer ungesund melden. Das ersetzt weder die WWDG-Fensterprüfung noch einen unabhängigen Hardware-Watchdog.